CUSTOMER ONBOARDING AI

The Claude Code Primitives Turning Point

From Deterministic Engineering → Harness Engineering → Probabilistic AI

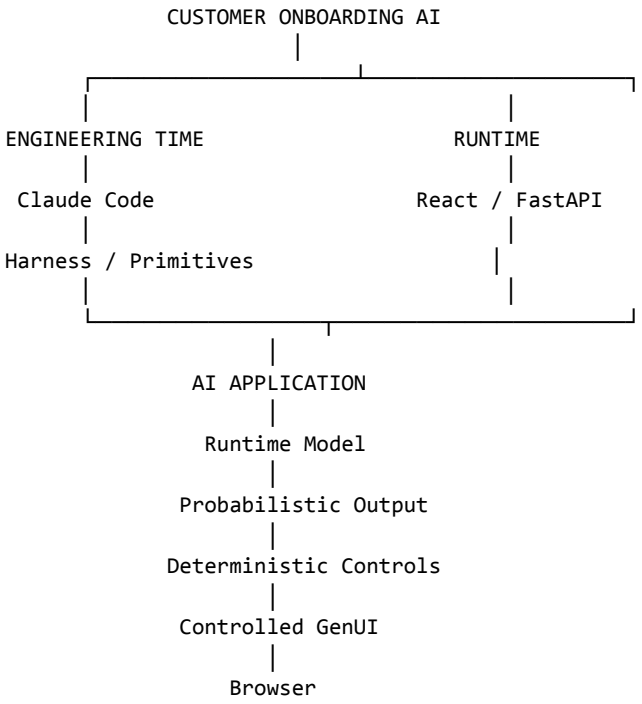
Enterprise-Grade Architecture & Learning Guide

Customer Onboarding AI • Tasks 1–6 completed • Task 6A proposed

1. Why This Is a Turning Point

The Customer Onboarding AI project has reached an important architectural transition. Tasks 1–6 deliberately established a deterministic foundation: a business problem, specification, machine-checkable UISpec contract, deterministic validator, controlled renderer, browser UI, and FastAPI Domain Runtime. The next stage is not simply 'add an AI model'.

The project can now become a continuous case study for two complementary ideas: how to build software with an AI coding agent inside a controlled engineering harness, and how to build a runtime application that safely handles probabilistic AI output.



2. The Six Claude Code Core Primitives

The project's primitive teaching material frames six core primitives: CLAUDE.md, Rules, Skills, Hooks, Plugins, and the Agent SDK. The source describes a primitive as a basic, atomic, composable, distinct, framework-provided building block. 'Primitive' describes the unit of thinking, not necessarily the internal complexity of the implementation.

#	Primitive	Core question	Role
1	CLAUDE.md	What should the agent know about this project?	Standing project knowledge
2	Rules	What focused behaviors should the agent always follow?	Focused behavioral guidance
3	Skills	What capabilities can the agent reach for on specific tasks?	On-demand capabilities
4	Hooks	What automatic actions should the harness take around tool use?	Harness-triggered automation/enforcement

5	Plugins	How do we package and share the above as one installable unit?	Distribution and packaging
6	Agent SDK	How do we build our own agents on top of all of this?	Programmatic agent interface

3. The Critical Distinction: Agent vs Harness

A central teaching point from the primitive material is who decides when something happens.

```
CLAUDE.md / Rules
        ↓
Agent's standing context and behavior

Skills
        ↓
Agent recognizes and invokes a capability

Hooks
        ↓
Harness recognizes an event and executes automatically

Plugins
        ↓
Package the capabilities and automation for distribution

Agent SDK
        ↓
Programmatic control for building custom agent systems
```

In particular, Hooks are not merely another Skill. The source material emphasizes that the agent decides when to use a Skill, whereas the harness fires a Hook deterministically around configured events.

4. What We Have Already Built

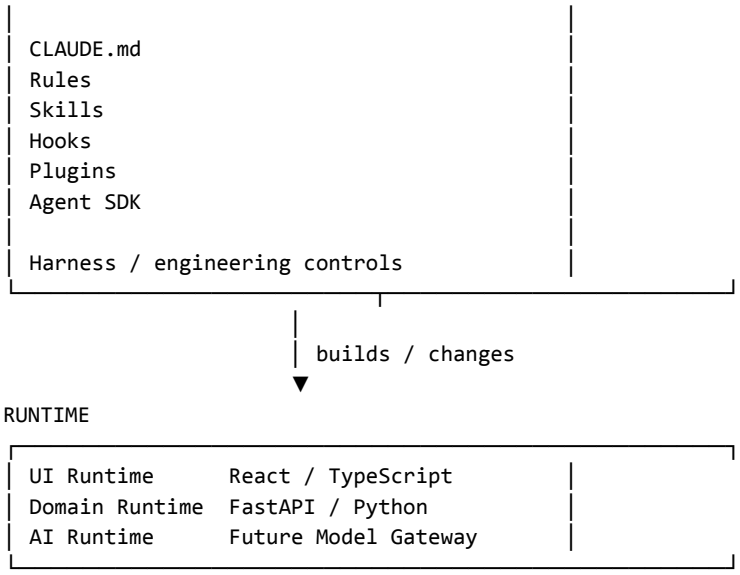
```
Task 1  Project Engineering Foundation              ✓
Task 2  UISpec Model Contract + JSON Schema            ✓
Task 3  Deterministic UISpec Validator                 ✓
Task 4  Controlled React Renderer                      ✓
Task 5  Minimal Runnable React UI                      ✓
Task 6  FastAPI Domain Runtime + API-backed UI         ✓
Task 6A ContextPayload Request Seam                    → next small seam
```

This matters because CLAUDE.md is already present. We are therefore not introducing the primitives from a blank slate; we are extending an existing engineering foundation.

5. Two Planes: Engineering Time and Runtime

The six primitives should not be confused with runtime application components. They primarily shape the engineering-time environment in which Claude Code operates.

```
ENGINEERING TIME
┌──────────────────────────────────────────────────┐
│ Claude Code                                      │
```



This separation prevents a common conceptual mistake: a Claude Code Hook is an engineering-harness mechanism, not an application API endpoint.

6. Primitive #1 — CLAUDE.md

CLAUDE.md is already part of the Customer Onboarding AI project. It provides durable project knowledge and establishes the architectural context Claude Code should understand.

```
CLAUDE.md
│
├── Business context
├── Architecture
├── Controlled GenUI principles
├── Approved component vocabulary
├── Security boundaries
├── Engineering scope
└── Project conventions
```

Teaching message: use CLAUDE.md for durable project knowledge, not as a dumping ground for every rule or procedure.

7. Primitive #2 — Rules

Rules should be introduced as focused behavioral guidance. Rather than putting every instruction into CLAUDE.md, separate specialized behaviors into scoped rules.

```
.claude/
└── rules/
    ├── security.md
    ├── testing.md
    ├── api.md
    └── ui.md
```

Examples for this project:

- Never introduce eval(), new Function(), or arbitrary dynamic component execution.

- Never bypass the controlled renderer.
- Every implementation increment must have deterministic tests.
- Run regression tests before a checkpoint.

Teaching distinction:

```
CLAUDE.md  = What should the agent know?
Rules      = What focused behavior should the agent follow?
```

8. Primitive #3 — Skills

Skills package reusable capabilities that the agent can recognize and invoke for particular tasks.

```
.claude/
└── skills/
    ├── implement-api-endpoint/
    │   └── SKILL.md
    ├── validate-uispec/
    │   └── SKILL.md
    ├── add-genui-component/
    │   └── SKILL.md
    └── run-quality-check/
        └── SKILL.md
```

Potential Customer Onboarding AI Skills:

- Implement an API endpoint according to the project's Domain Runtime conventions.
- Update a UISpec contract and its deterministic tests in the correct order.
- Add an approved GenUI component without bypassing the registry.
- Run the project's quality and regression checks.

Teaching distinction:

```
Rule  = behavior
Skill = capability / procedure
```

9. Primitive #4 — Hooks

Hooks are the transition from agent guidance to harness-controlled automation. The harness triggers them around events such as tool use. The agent does not decide whether the configured event hook fires.

```
Claude edits a TypeScript file
          ↓
     PostToolUse Hook
          ↓
     TypeScript / formatting / checks

Claude attempts a prohibited operation
          ↓
     PreToolUse Hook
          ↓
        BLOCK
```

Customer Onboarding AI examples:

- Run a relevant test after a source-file change.

- Run schema verification after UISpec/schema changes.
- Detect prohibited executable-code mechanisms.
- Block dangerous tool operations.

Teaching distinction:

```
Skill → agent decides to use it
Hook  → harness decides when it fires
```

10. Primitive #5 — Plugins

Plugins solve the distribution problem. Once Rules, Skills, Hooks and related commands/configuration form a coherent toolkit, they can be packaged as a versioned, installable unit.

```
customer-onboarding-ai-plugin/
├── .claude-plugin/
│   └── plugin.json
├── skills/
├── hooks/
├── scripts/
├── commands/
└── README.md
```

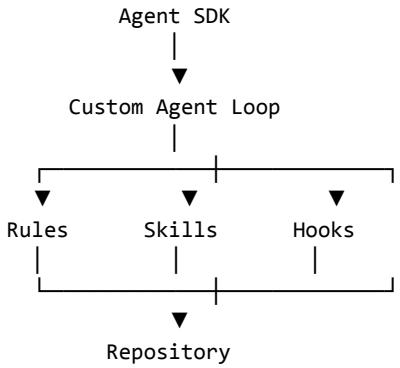
Potential enterprise bundle:

- UISpec validation Skill
- API implementation Skill
- Security Hooks
- Quality/regression Hooks
- Checkpoint slash command
- Documentation and usage guidance

The teaching value is that learners experience the progression from individual primitives to a coherent organizational engineering capability.

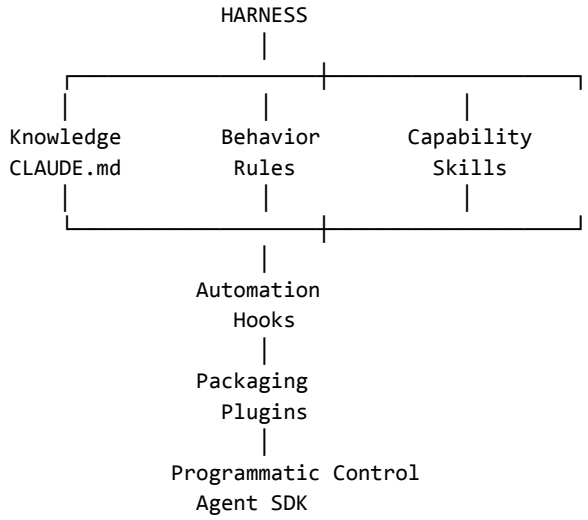
11. Primitive #6 — Agent SDK

The Agent SDK belongs later in the progression. It becomes meaningful after learners understand the primitives and the harness they compose.



This lets the project move from a human-driven Claude Code workflow toward a programmatic agentic engineering workflow, while retaining the same project controls.

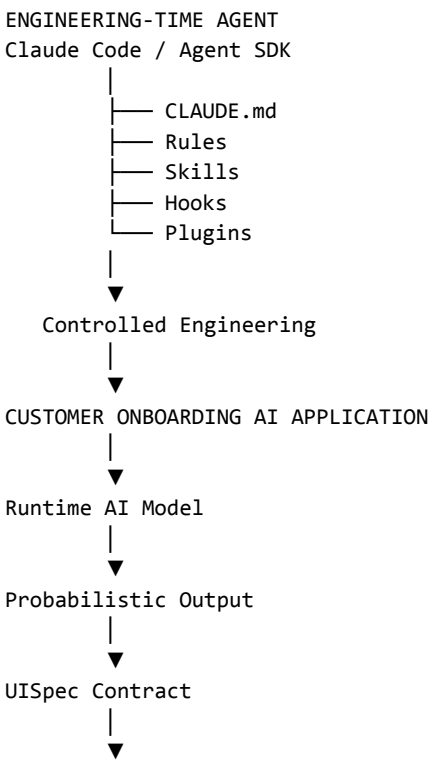
12. How the Six Primitives Become a Harness

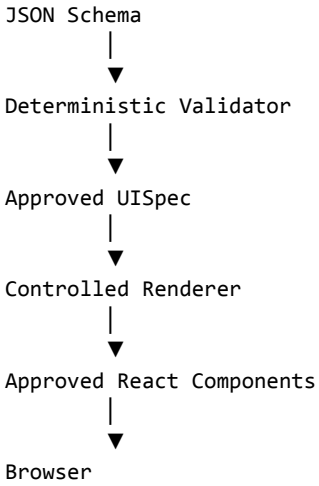


The harness should be taught as the broader controlled environment around the agent, rather than as a seventh primitive. The six primitives provide composable mechanisms; the harness is the controlled system in which those mechanisms operate.

13. The Connection to Probabilistic Runtime AI

This is the central architectural connection. Engineering-time harness controls and runtime AI controls solve different problems, but they reinforce each other.





The key idea is that there are two control surfaces: control over how AI-assisted software is built, and control over how probabilistic model output is allowed to affect the running application.

14. Two Different Kinds of Determinism

Layer	Question	Example
Engineering harness	How do we control the agent's development behavior?	Rules, Hooks, Skills, Plugins
Runtime application	How do we control probabilistic model output?	JSON Schema + deterministic validator + renderer

15. Revised Project Roadmap

```
COMPLETED
──────────────────────────────────────────────────
Task 1   Project foundation                    ✓
Task 2   Model contract + JSON Schema          ✓
Task 3   Deterministic validator               ✓
Task 4   Controlled renderer                   ✓
Task 5   Runnable React UI                     ✓
Task 6   FastAPI Domain Runtime                ✓
──────────────────────────────────────────────────

NEXT CONTEXT SEAM
──────────────────────────────────────────────────
Task 6A  ContextPayload request seam           →

AI-NATIVE ENGINEERING LAYER
──────────────────────────────────────────────────
Task 7   Rules
Task 8   Skills
Task 9   Hooks
Task 10  Plugin packaging
Task 11  Agent SDK / agentic engineering loop

RUNTIME AI LAYER
──────────────────────────────────────────────────
Task 12  Model Gateway
Task 13  Runtime model → UISpec
Task 14  AI Evals
```

```
Task 15  Feedback loop
Task 16  Continuous improvement
Task 17  Production hardening
```

This roadmap intentionally avoids jumping directly from FastAPI to a model API. The engineering harness should first become visible and demonstrable.

16. Why Task 6A Still Comes First

```
Task 6
FastAPI
   ↓
Fixed developer-authored fixture

Task 6A
ContextPayload
   ↓
Developer-authored fixture
   ↓
Validator

Future runtime AI
ContextPayload
   ↓
Model Gateway
   ↓
Raw model output
   ↓
Validator
```

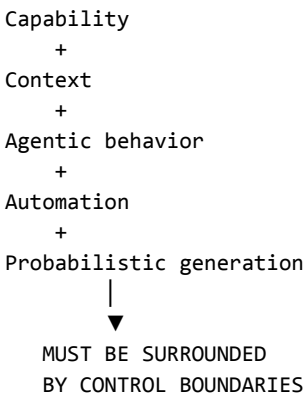
Task 6A makes the runtime context explicit without introducing the model. It creates a clean seam that the future model gateway can occupy.

17. Recommended Teaching / Implementation Sequence

1. Complete and checkpoint Task 6A.
2. Introduce Rules using Customer Onboarding security and testing behaviors.
3. Introduce Skills using reusable UISpec/API engineering capabilities.
4. Introduce Hooks to make quality and security actions harness-driven.
5. Package the reusable Rules/Skills/Hooks into a Customer Onboarding engineering Plugin.
6. Introduce the Agent SDK to demonstrate a programmatic agent loop.
7. Only then introduce the runtime Model Gateway.
8. Connect ContextPayload to the model and produce raw structured model output.
9. Send model output through the existing deterministic validator.
10. Introduce AI evaluations and a feedback loop.

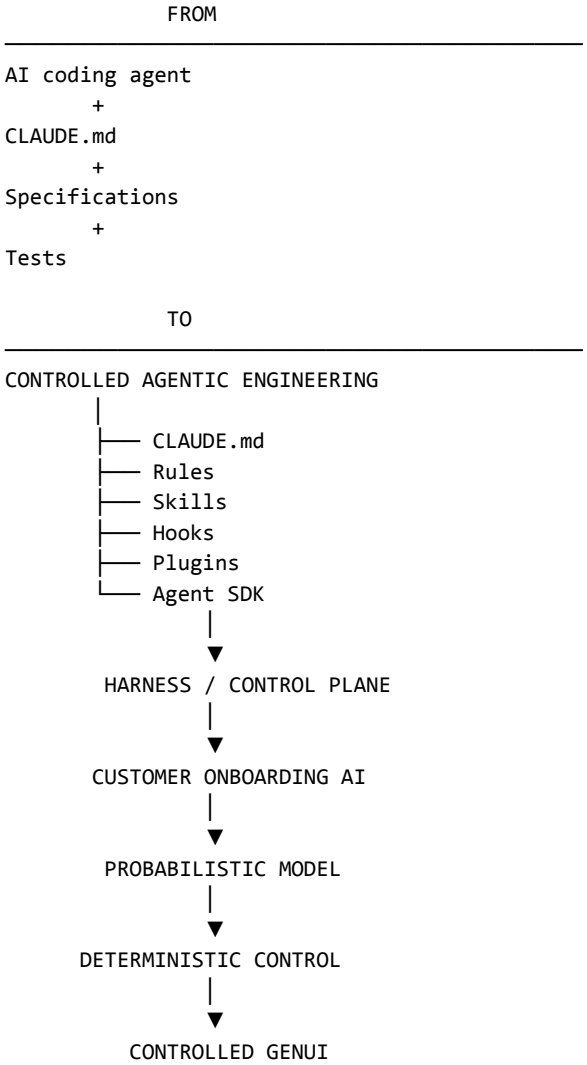
18. The Core Enterprise Lesson

The objective is not to make the Customer Onboarding AI system 'more autonomous' as quickly as possible. The objective is to increase capability while preserving control.



This is why the project deliberately separates agentic engineering controls from runtime AI controls.

19. Turning-Point Mental Model



20. Final Architectural Principle

AI can propose. The system decides what is allowed.

At engineering time, the harness controls how the coding agent operates. At runtime, the application controls what the probabilistic model is permitted to cause. Together, these two control planes form the foundation for an enterprise-grade AI-native engineering approach.

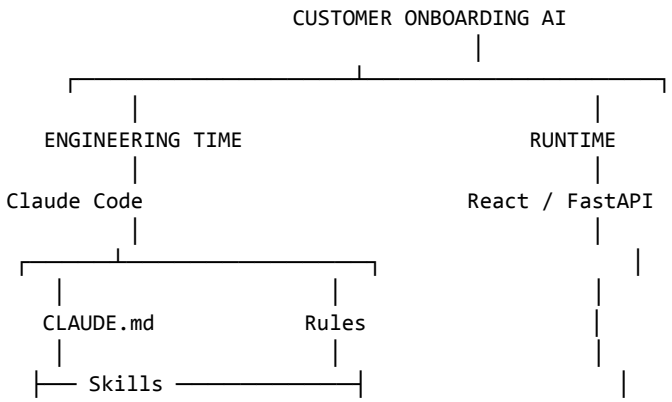
21. Status at This Turning Point

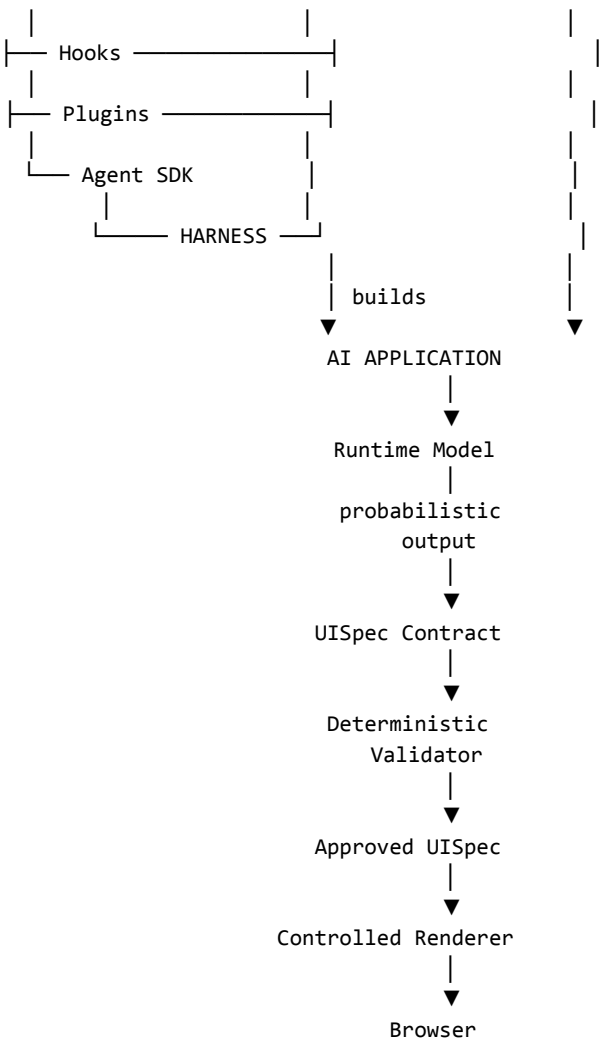
Area	Status
CLAUDE.md	Already implemented
Rules	Next AI-native engineering primitive
Skills	Planned
Hooks	Planned
Plugins	Planned
Agent SDK	Planned
Harness engineering	Emerging through the composition of primitives
UISpec contract	Implemented
Deterministic validator	Implemented
Controlled renderer	Implemented
FastAPI Domain Runtime	Implemented
ContextPayload seam	Next runtime seam
Runtime model gateway	Future
AI evaluation	Future
Feedback loop	Future

This document is an architectural turning-point record. It should guide the next implementation increments without collapsing the engineering-time harness and runtime AI layers into one concept.

Integrated Customer Onboarding AI Architecture

The following architecture brings the Claude Code primitives, the engineering-time harness, and the runtime probabilistic AI control path into one view. It distinguishes the engineering-time control plane from the runtime application path.





How to read the diagram: the left side represents the engineering-time control plane where Claude Code and its primitives shape how the application is built. The right side represents the runtime application. The harness builds the AI application, while the runtime model remains probabilistic. Its output is constrained by the UISpec contract, checked by deterministic validation, and only then passed to the controlled renderer.

Core principle: AI can propose. The system decides what is allowed.